

# Dafny 从零精讲：第 8–13 讲（证明 · 量词 · 归纳 · 加强归纳假设）

配套课程：Foundations of Computing (COMP1600/6260), Michael Norrish 本教程覆盖：Lecture 08–13 的全部 PPT 内容 + code 文件夹里对应的练习文件。写作原则：**不跳步**。每一个结论都会说明”这一步凭什么成立”。

## 本教程怎么用（先读这段）

你的三个问题分别对应：

| 你的问题           | 对应章节                                                                |
|----------------|---------------------------------------------------------------------|
| ① 证明题部分        | 第一部分（云与 assert/调用引理）、第二部分（案例拆分）、第五部分（归纳）、第六部分（加强归纳假设）——证明是一条主线，按顺序读 |
| ② 第 13 节没听     | 第六部分，完整重讲了一遍，前面的部分（尤其第五部分归纳）是它的基础                                   |
| ③ “存在”和相关”关键字” | 第三部分（forall）、第四部分（exists）；所有关键字的速查表在第七部分                            |

建议读法：**按部分顺序读**。第 8、9 讲是地基（怎么写证明），第 10、11 讲是量词（forall / exists 的各种”面孔”），第 12 讲是归纳，第 13 讲是”加强归纳假设”——难度是递增的，但每一级只比前一级多一个新概念。

阅读约定：- 所有代码块都是 Dafny 代码，可以直接粘进 VS Code（装了 Dafny 插件）验证。- 标注 **【练习】** 的小节是老师课上跳过或没讲完的题，我给了完整解法。个别标注了”需在你机器上验证”，因为我这边没装 Dafny 编译器，逻辑上逐行推过，但没跑过机器验证；如果报红，把报错发我，我们再调。- 标注💡的是”为什么”，考试和写作业最容易被问。

## 第〇部分：十分钟最小背景

后面的证明教程只依赖这些概念。如果你已经会了可以跳过，但“函数 vs 引理”那一小节不要跳。

### 0.1 列表类型 list 和 match

Dafny 里列表是这么定义的（第 4 讲起反复用）：

```
datatype list<T> = Nil | Cons(T, list<T>)
```

拆开说：- datatype = “定义一种新数据形状”，它列出这种数据**所有可能的样子**。- list<T>：尖括号 T 是”元素类型”的占位符（泛型）。list<int> 就是整数列表。- 一个列表**只有两种可能**：- Nil：空列表，什么都不含；- Cons(h, t)：非空列表，h 是头（第一个元素），t 是尾（一个更短的列表）。- 例子：[3, 5] 在 Dafny 里写作 Cons(3, Cons(5, Nil))。

match 就是”看看它是哪种样子，分别处理”：

```
function length<T>(l: list<T>): nat {
  match l
  case Nil          => 0           // 如果是空的, 长度是 0
  case Cons(h, t) => 1 + length(t) // 如果非空, 长度 = 1 + 尾巴的长度
}
```

- `nat` 是”自然数”类型 (0, 1, 2, ...)。 `int` 是所有整数。
- 访问字段可以写点号: 若 datatype `list<T> = Nil | Cons(h: T, t: list<T>)` (字段有名字), 则 `l.h`、`l.t`。
- 判断”它是不是 `Cons` 造的”可以写 `l.Cons?` (带问号, 返回 `bool`); `l != Nil` 也行。

## 0.2 逻辑运算符 (Dafny 的”命题语言”)

| 写法                            | 意思                                                  | 例子                                  |
|-------------------------------|-----------------------------------------------------|-------------------------------------|
| <code>== != &lt; &lt;=</code> | 相等/不等/大小                                            | <code>x == 3</code>                 |
| <code>&amp;&amp;</code>       | 且                                                   | <code>p &amp;&amp; q</code>         |
| <code>\ \ </code>             | 或                                                   | <code>p \ \  q</code>               |
| <code>!</code>                | 非                                                   | <code>!p</code>                     |
| <code>==&gt;</code>           | 蕴含 (如果...那么...)                                     | <code>x &gt; 0 ==&gt; foo(x)</code> |
| <code>&lt;==&gt;</code>       | 当且仅当 (互相推出)                                         | <code>A &lt;==&gt; B</code>         |
| <code>a &lt; b &lt; c</code>  | 连写不等式, 等于 <code>a &lt; b &amp;&amp; b &lt; c</code> | <code>lo &lt; x &lt; hi</code>      |

💡 `==>` 和 `<==>` 的区别是证明题的命根子: `P ==> Q` 只说”`P` 真时 `Q` 必真”; `P <==> Q` 说了两件事: `P ==> Q` 且 `Q ==> P`。Dafny 证 `<==>` 时要两头都证 (自动或手动)。

## 0.3 函数 `function`、谓词 `predicate` 与 `ghost`

```
function add2(x: int): int { x + 2 } // 返回一个值
predicate isBig(x: int): bool { x > 100 } // 返回真/假的函数, 习惯上叫”谓词”
```

- 它们的共同点: **必须有明确的定义**, Dafny 会检查递归函数一定终止 (第 7 讲内容: Dafny 自动看”参数是不是在变小”, 看不见时你要用 `decreases` 提示)。
- `ghost` 关键字: 标记一个东西”只用于推理、不参与运行”。第 11 讲的 `ghost predicate divides` 用了它——因为谓词体里写了 `exists` (存在量词), 机器没法”计算”它, 只能推理。`ghost` 函数可以被其他证明 (`ghost` 代码) 调用, 但不能在普通可执行代码里调用。
- **抽象谓词**: `predicate foo(n: nat)` 不带大括号体 = “我不告诉你 `foo` 是什么, 只知道它存在”。相当于数学里的”设 `P` 是某个性质”。第 10 讲的 `simplest-quantifiers.dfy` 就这么干。

## 0.4 引理 `lemma`——证明的载体, 和函数有本质区别

```
lemma simple(m: nat, n: nat)
  requires m <= n           // 假设 (前提): 可以随便用
  ensures 2 * m <= 2 * n    // 目标: 必须被证明
{}                          // ← 大括号里是”证明过程”, 空 = Dafny 自己能搞定
```

逐行读法: - `lemma` 定义一个**数学命题**加上它的证明。- `m, n` 是任意的自然数 (这里藏着全称量词, 第三部分详细讲)。- `requires ...`: 使用这个引理时需要满足的前提; 在引理**内部**, 它们是**已经成立的假设** (白拿的)。- `ensures ...`: 引理内部是**待证的目标**; 引理外部是**拿来用的结论**。- `lemma` 没有返回类型。它不产出值, 只产出”一条已被证明的真理”。

💡 **函数 vs 引理——最重要的区别：** - function 造**值**（数字、列表……），可以被程序运行、被别的函数调用。 - lemma 造**真理**（一条证完的命题），只能被**证明里**”调用”（见 1.5）。 - 引理里**不许**用赋值  $x := e$  和循环 while（因为它们改变状态，属于”语句”范畴），但可以用 if/else、match——它们在证明里的用法是”分情况讨论”，第二部分细讲。

## 0.5 lset：这门课把”集合”用”列表”来模拟

第 8、9 讲的文件里：

```
datatype lset = Nil | Cons(int, lset)    // lset = 用链表冒充的集合
type lset = list<int>                  // 第 10 讲文件里用"类型别名"写法
```

两种写法都出现过，效果一样：lset 就是整数列表。老师故意用列表冒充集合，是为了练习”自己定义的东西要自己证明它 behaving 正确”——比如后面会看到”列表当集合用”时的子集、并集证明。

---

## 第一部分：证明是怎么运作的（Lecture 08）

对应代码：code/lecture08-dafny-code/lecture08/ 下四个文件： lemma-parts.dfy、assert-proofs.dfy、setlists.dfy、chunks-terminates.dfy

### 1.1 核心心智模型：乌云与地面（the “Cloud”）

第 8 讲 PPT 用了个贯穿全课的比喻，记住它你就懂 Dafny 证明的一切：

⚡ ← “云”：所有已知事实（requires + 你 assert 的 + 调用引理换来的）  
⚡  
\_\_\_\_\_ ← “地面”：ensures 目标

- 证明 = 让闪电从云打到地面。云里没有足够的事实连到地面时，Dafny 就报红。
- 你帮助 Dafny 的唯一方式，就是往云里加新事实。加事实只有两种动作：
  1. assert  $\phi$ ; —— “ $\phi$  这条事实我宣布成立”，但 Dafny 会检查  $\phi$  确实能从现有云推出；
  2. 调用一条已证的引理 foo(实参); —— 把 foo 的 ensures（代入实参后）整条塞进云。
- 空证明 {} 能过的情况只有两种：**ensures** 和某个 **requires** 长得几乎一样（比如  $m \leq n$  推  $2*m \leq 2*n$ ，这种小算术跳跃 Dafny 会自动做）；或者 **Dafny 靠函数定义+自动归纳就能推过去**。

先看一个 Dafny 自动通过的例子：

```
lemma simple(m: nat, n: nat)
  requires m <= n
  ensures 2 * m <= 2 * n
  {}
```

为什么空着也行：云里有  $m \leq n$ ，目标是  $2m \leq 2n$ 。这不是逐字相同，但差距小到 Dafny 的自动算术推理能一步跨过去。💡 差距大了（比如需要用到某条函数定义展开、需要归纳），空 {} 就会红，那就是你要动手的时候。

### 1.2 【动手实验】lemma-parts.dfy——引理的解剖台

这个文件不是难题，是老师让你”拆着玩”的沙盒。当前内容：

```
lemma simple(m: real, n: real)
  requires m <= n
```

```

requires n < m
ensures m + 1000.0 < m
{}

```

注意它同时假设了  $m \leq n$  和  $n < m$ ——这两条不可能同时成立（没有任何实数满足）。这时云是”自相矛盾”的。逻辑学有一条铁律（爆炸原理）：**前提是假的，任何结论都算”对”**（“如果  $1+1=3$ ，那我就是教皇”这种句子在逻辑上是真命题）。所以 Dafny 照样绿灯——它先发现前提根本凑不齐，目标就不用证了。

PPT 里列的实验（推荐每个都试 5 分钟，比看十遍教程有用）：

| 实验                                   | 结果和原因                                                               |
|--------------------------------------|---------------------------------------------------------------------|
| 给 lemma 加返回类型：lemma simple(...): int | <b>报错</b> 。lemma 不是函数，它不返回值，只返回”证明”。                                |
| 删掉一个 requires                        | 只剩 $m \leq n$ 时， $m + 1000.0 < m$ 证不出来，变红——云不够厚，闪电打不到地面。            |
| 两个 requires 都删                       | 更红。没有任何假设时 $m+1000 < m$ 对实数恒假。                                      |
| 把 ensures 删掉                         | 合法但没意义（没有目标 = 没有要证的东西，也= 没人在将来能用它）。                                 |
| 把参数 m, n 删掉                          | 前提和目标里的 m, n 变成未定义名字，直接报错。这也说明 lemma 的参数就是”被任意 ( $\forall$ ) 定的变量”。 |

### 1.3 招式一：assert

```
assert  $\phi$ ;
```

意思是：“Dafny，请检查  $\phi$  能从云里推出来；如果推得出， $\phi$  从此也进云。”所以 assert 有双重身份：既是一个”自查”（这一行红 = 你推不过去），又是一块”垫脚石”（通过后云变厚，后面的目标更好打）。

#### 例 1：solve\_quadratic——自动过关

```

lemma solve_quadratic(x: int)
  requires x * x + 4 * x - 12 == 0
  ensures x == 2 || x == -6
{}

```

云里是方程  $x^2 + 4x - 12 = 0$ ，目标是” $x = 2$  或  $x = -6$ ”。Dafny 的自动求解器对一元二次方程够用（因式分解  $(x-2)(x+6)$  这种它自己会凑），所以空 {} 就过。

#### 例 2：solve\_poly——三次方程，自动不行了，assert 递刀

```

lemma solve_poly(x: int)
  requires x * x * x + x * x - 24 * x + 36 == 0
  ensures x == 3 || x == 2 || x == -6
{
  assert x * x * x + x * x - 24 * x + 36 == (x - 3) * (x - 2) * (x + 6);
}

```

为什么自动不行：三次乘积超出 Dafny 自动因式分解的能力（PPT 原话：“moving to cubics pushes past whatever automation was handling the quadratic”）。为什么这一行 assert 就行，拆开看：1. 人（你）替机器做了因式分解这一步： $x^3 + x^2 - 24x + 36 =$

$(x-3)(x-2)(x+6)$ 。2. `assert` 让 Dafny 验证这条恒等式——两边展开后是同一个多项式，**逐项乘开对比**这种是机械活，Dafny 会做。3. 通过后云里多了  $x^3+x^2-24x+36 == (x-3)(x-2)(x+6)$ 。再结合原有假设  $x^3+x^2-24x+36 == 0$ ，代换得  $(x-3)(x-2)(x+6) == 0$ ；三个整数乘积为 0  $\rightarrow$  至少一个因子为 0  $\rightarrow x == 3 \vee x == 2 \vee x == -6$ ，正好是目标。闪电落地。

💡 这个模式叫“**人负责洞察，机器负责检验**”。Dafny 证明的日常 workflow 就是：自动不行  $\rightarrow$  想一个”差一步就能连上”的小事实  $\rightarrow$  `assert` 它。

### 例 3: `xltdiv`——把整除的定义知识 `assert` 出来

```
lemma xltdiv(x: nat, y: nat, z: nat)
  requires 0 < z
  ensures x < y / z <==> (x + 1) * z <= y
{
  assert y == (y / z) * z + y % z;
}
```

- Dafny 里  $y / z$  是**向下取整**的整数除法， $y \% z$  是余数 ( $0 \leq y \% z < z$ )。它们的关系由一条恒等式锁死： $y = (y/z) \cdot z + y \% z$  (“被除数 = 商  $\times$  除数 + 余数”)。Dafny 内置知道这条，但常常需要你**主动 `assert` 出来进云**，它才会用。
- 有了它： $x < y/z \Leftrightarrow$  (两边乘正数  $z$ ，注意  $z > 0$  由 `requires` 保证)  $\Leftrightarrow x \cdot z < (y/z) \cdot z \leq (y/z) \cdot z + y \% z = y$  取整性质一路推  $\Leftrightarrow (x+1) \cdot z \leq y$  (整数上  $a \cdot z < b \cdot z$  的下一个**就是**  $(a+1)z \leq b$ )。这些整数跳跃 Dafny 拿到余数恒等式后可以自动收尾。
- 💡 为什么 `requires` 里有  $0 < z$ ：除数为零时除法无意义；这是”引理的前提在证明中被真实用到”的一个例子——你以后写任何带  $/$  的命题都要配  $0 < \text{除数}$ 。

## 1.4 招式二：调用引理（把已证的东西当砖搬）

```
lemma mytheorem(x1:  $\tau_1$ , ..., xn:  $\tau_n$ )
  requires  $\phi$ 
  ensures  $\psi$ 
{ ... }
```

// 在别的证明里：

```
mytheorem(t1, ..., tn); // 云里立刻多一条： $\psi$  里的  $x1..xn$  换成  $t1..tn$ 
```

调用引理 = **把那条引理的结论（代入具体实参后）免费塞进云**。代价：调用点必须同时证明实参满足那条引理的每条 `requires` (Dafny 在调用那行标红提醒你)。

### 完整故事：sum100——第 5、7 讲老朋友

```
function sumUpTo(m: nat): nat {
  if m == 0 then 0 else m + sumUpTo(m - 1)
}
```

```
lemma triangle(n: nat)
  ensures sumUpTo(n) == n * (n + 1) / 2
{ } // 归纳法自动完成（第五部分详解）
```

```
lemma sum100()
  ensures sumUpTo(100) == 5050
{ triangle(100); }
```

逐帧播放 `sum100` 里发生了什么：1. 一开始云里只有 `sumUpTo` 的定义。Dafny 证 `sumUpTo(100) == 5050` 时只会”展开定义”：展开 100 次太贵，展开几步又不够，红。2. `triangle(100)`；这行调用：`triangle` 的 `ensures` 是 `sumUpTo(n) == n(n+1)/2`， $n$  代换成 100，

云里多了一条  $\text{sumUpTo}(100) == (100 \times 101) / 2$ 。3. 目标  $\text{sumUpTo}(100) == 5050$  与云里新事实合并，只差算  $(100 \times 101) / 2$  是不是 5050——纯常数算术，Dafny 秒杀。绿灯。

💡 这就是引理复用的全部意义：你证明一次通用事实（对任意  $n$ ），之后每个具体场景调用一次就领一次现成的结论。

### setlists.dfy: member\_union 为什么要绕一步

```
datatype lset = Nil | Cons(int, lset)
```

```
function append(l: lset, k: lset): lset {           // 列表串接
  match l
  case Nil      => k
  case Cons(h, t) => Cons(h, append(t, k))
}
```

```
predicate member(i: int, A: lset) {                // i 是否在列表里
  match A
  case Nil      => false
  case Cons(j, r) => i == j || member(i, r)
}
```

```
function union(A: lset, B: lset): lset { append(A, B) } // 集合"并"= 列表串接
```

```
lemma member_append(x: int, A: lset, B: lset)
  ensures member(x, append(A, B)) <==> member(x, A) || member(x, B)
{} // Dafny 自动归纳能证（对 A 归纳）——现在先接受这个事实，第五部分解释原理
```

```
lemma member_union(x: int, A: lset, B: lset)
  ensures member(x, union(A, B)) <==> member(x, A) || member(x, B)
{
  member_append(x, A, B); // ← 只此一行
}
```

- 直接写空 {} 证 member\_union, Dafny 是红的 (PPT: “dafny: sad; user: sadder”)。差别只在 union 这层定义外衣。
- 补救：先证不穿外衣的版本 member\_append (它能自动过)，再在 member\_union 里调用它。调用后云里是  $\text{member}(x, \text{append}(A, B)) <==> \dots$ ，而目标  $\text{member}(x, \text{union}(A, B)) <==> \dots$ ：Dafny 自己会把  $\text{union}(A, B)$  按定义展开成  $\text{append}(A, B)$ ，两边对上了，绿灯。
- 💡 经验法则：当 Dafny 因为“你的概念穿了层定义外衣”而卡住时，先对“裸版本”证明，再调用 + 让它展开外衣。这是 Dafny 里最高频的破红技巧之一。

### requires 躲不掉的账

PPT 的反面教材 (第 24–26 页):

```
lemma super_useful(x: int) requires 2 < x ensures ...
```

```
lemma even_more_useful(i: int, j: int) ... {
  super_useful(i + 2 * j); // 报错! Dafny 在这里要求你证明 2 < i + 2*j
}
```

调用带前提的引理 = 在调用点立刻还债。有人想偷懒，把前提搬进结论里做蕴含：

```
lemma maybe_useful(x: int)
  ensures 2 < x ==> ... // 没有 requires, 调用永远"免费"
```

PPT 的结论 (“Work Cannot Be Escaped”): 活不会消失, 只会转移。免费的蕴含版在你真正需要结论  $\psi$  的那一天, 还是得自己先证出前件  $\phi$ 。而 `requires` 版本让 Dafny 在调用处主动提醒你 (红色波浪线)。所以工程习惯: 该要前提就写 `requires`, 别塞进 `==>` 里。只有一种例外值得用蕴含: 前提显然成立、或者你故意想把”分情况”推迟到证明末尾——后面第 10 讲 dumb 例子就会用到蕴含。

## 1.5 招式三: `assert ... by { ... }`——给证明搭抽屉

假设一段证明需要 6 步, 但只有最后 2 步是”思想”, 前 4 步是”体力”:

```
assert P1; assert P2; assert P3;
sub_lemma(e1, e2, e3);
assert P4;
assert P5;
assert P6;    // 全部堆在一起, 人和 Dafny 都眼花
```

Dafny 提供”嵌套证明”:

```
assert P5 by {
  assert P1; assert P2; assert P3;
  sub_lemma(e1, e2, e3);
  assert P4;
}          // ↑ 这 4 步只为证 P5, 证完就收进抽屉
assert P6; // 外层云里依然有 P5, 继续
```

- 语法事实 (PPT 原话): “语句 + 表达式”是合法表达式。`assert X;` 别的表达式这种写法合法, 意思是”先跑一遍证明语句, 再取值”。这个语法在下面 chunks 的证明里直接被用到。
- `by` 的好处: 对人类可读; 对 Dafny 是收窄注意力——它只需要在抽屉里证 P5, 不用同时盯着全局目标。

## 1.6 压轴例: 证明 `chunks` 能终止 (`chunks-terminates.dfy`)

这是第 7 讲留的坑 + 第 8 讲的填坑, 把前面所有招式用全了。

### 问题背景

```
datatype list<T> = Nil | Cons(T, list<T>)
function length<T>(l: list<T>): nat {
  match l
  case Nil          => 0
  case Cons(h, t) => 1 + length(t)
}
function take<T>(n: nat, l: list<T>): list<T> {    // 取前 n 个
  match l
  case Nil          => Nil
  case Cons(h, t) => if n == 0 then Nil else Cons(h, take(n - 1, t))
}
function drop<T>(n: nat, l: list<T>): list<T> {    // 扔掉前 n 个
  match l
  case Nil          => Nil
  case Cons(h, t) => if n == 0 then l else drop(n - 1, t)
}

function chunks<T>(n: nat, l: list<T>): list<list<T>>
  requires 0 < n
{
  match l
```

```

case Nil => Nil
case _   => Cons(take(n, l), chunks(n, drop(n, l))) // 每 n 个切一块
}

```

`chunks(2, [a,b,c,d,e])` 应产出 `[[a,b],[c,d],[e]]`：切下前 `n` 个，再把剩下的继续切。逻辑完全合理，但 **Dafny 拒绝接受**：它要检查递归调用一定会停。Dafny 默认的”变小”判定看的是**参数本身**——第二个参数从 `l` 变成 `drop(n, l)`，Dafny 看不出 `drop(n,l)` 比 `l` “短”（这需要一条关于 `drop` 的定理，它不会自己发明）。

### 解决方案：三件套

```

lemma length_drop<T>(n: nat, l: list<T>)
  ensures length(drop(n, l)) == if n >= length(l) then 0 else length(l) - n
{}

```

先把”`drop` 之后长度怎么变”这条定理立起来（空 {}：对 `l` 自动归纳能过，第五部分解释）。直观读法：从长度为 `L` 的列表 `drop` 掉 `n` 个，若  $n \geq L$  剩 0 个，否则剩  $L - n$  个。

```

function chunks<T>(n: nat, l: list<T>): list<list<T>>
  requires 0 < n
  decreases length(l) // ② 告诉 Dafny：终止性看 length(l) 是否变小
{
  match l
  case Nil => Nil
  case _ =>
    assert length(drop(n, l)) < length(l) by { length_drop(n, l); } // ③
    Cons(take(n, l), chunks(n, drop(n, l)))
}

```

② `decreases length(l)`：度量函数。意思是”每次递归我必须证明 `length(l)` 严格变小，且自然数不可能无限变小，所以会停”。③ 那行 `assert` 就是在完成这个义务，逐层拆开：- 要证：`length(drop(n,l)) < length(l)`。- `by` 抽屉里调用 `length_drop(n, l)`，云里得到：`length(drop(n,l)) == if n >= length(l) then 0 else length(l) - n`。- 分两路，两路都 `< length(l)`：- 若  $n \geq \text{length}(l)$ ：`drop` 结果是空列表。此时 `l` 本身长度  $\leq n$ 。注意进入本分支的前提：`l` 不是 `Nil` (`Nil` 在上一 `case` 处理了)，所以  $\text{length}(l) \geq 1 > 0 \checkmark$ 。Dafny 能自己把”非 `Nil` 的列表长度  $\geq 1$ ”这一步推出来 (`match` 过 `Nil` 和 `Cons`，`Cons` 的定义摆在那)。- 若  $n < \text{length}(l)$ ：结果是 `length(l) - n`，配合 `requires 0 < n` 得 `length(l) - n < length(l)`  $\checkmark$ 。- 于是 `assert` 通过，`decreases` 义务完成，函数被接受。

💡 `assert 事实 by { 证明 }` 出现在**函数体表达式**里的巧妙之处就是上面 ③：表达式中间能插证明语句（1.5 的”语法事实”）。

## 1.7 第一部分练习地图

| 文件                    | 状态  | 你会做什么                                        |
|-----------------------|-----|----------------------------------------------|
| lemma-parts.dfy       | 沙盒  | 说出 requires/ensures/参数各是干嘛的；解释”矛盾前提为何绿灯”     |
| assert-proofs.dfy     | 已全解 | 能自己给 solve_poly 找到 assert 的那条恒等式（提示：先手算因式分解） |
| setlists.dfy          | 已全解 | 解释 member_union 为什么先要 member_append          |
| chunks-terminates.dfy | 已全解 | 遮住代码，复述三件套：定理 + decreases + assert by        |

## 第二部分：分情况讨论（Lecture 09）

对应代码：code/lecture09-dafny-code/lecture09/ — subset-casesplits.dfy、all\_distinct\_nub.dfy

### 2.1 本讲口号（背下来，全课通用）

PPT 原话：“proofs about functions have structures **mimicking** those functions’ structure.” 关于一个函数的证明，结构要模仿这个函数本身的结构。

- 函数对布尔值分了 if/else → 证明里就 if P { } else { };
- 函数对列表 match 了 Nil/Cons → 证明里就 match l case Nil => ... case Cons(h,t) => ...

这个口号是第 12 讲归纳法的种子：函数怎么递归，证明就怎么归纳。

### 2.2 证明里的 if 到底给云加了什么

PPT 第 21 页的关键图景。写：

```
if P {  
  assert  $\phi_1$ ;  
  lemma_call(t);  
} else {  
  assert  $\phi_2$ ;  
}
```

Dafny 得到的不是”  $\phi_1$  成立”，而是三条带条件的事实：

- $P \implies \phi_1$
- $P \implies \text{lemma\_call 的 ensures}$
- $\neg P \implies \phi_2$

也就是说：在 if 的花括号内部，你白拿假设 P；在 else 内部，白拿假设  $\neg P$ ；出了 if/else 之后，这些局部假设收回，只剩下带蕴含的三条。这就是”分情况讨论”的精确逻辑含义：每种情况里目标都成立 → 目标总体成立。

## 2.3 证明里的 match 给云加什么

```
match l
case Nil      => { ... }
case Cons(h, t) => { ... }
```

- 在 case Cons(h,t) 内部：白拿假设  $l == \text{Cons}(h, t)$  (于是  $\text{length}(l) \geq 1$ 、 $l.t == t$ 、 $h$  就是第一个元素等，全部可用)；
- Dafny 知道列表只有这两种形状 (datatype 的封闭性：不是 Nil 就必是某个 Cons，不是某个 Cons 就必是 Nil)，所以两个 case 各自证完目标，整体目标就成立。

💡 这就是为什么 match 不需要写 default 分支——类型把所有可能性都列光了。

## 2.4 热身：subset\_trans (PPT 第 12 页 + 文件里已完整)

先复习主角定义 (subset-casesplits.dfy 开头)：

```
predicate subset(A: lset, B: lset) {
  match A
  case Nil      => true // 空集是任何集合的子集
  case Cons(x,xs) => member(x,B) && subset(xs,B) // 头在B里，且剩余也在B里
}
```

要证  $A \subseteq B \wedge B \subseteq C \Rightarrow A \subseteq C$ 。Dafny 版 (老师的完整证明)：

```
lemma subset_member(x: int, A: lset, B: lset) // 先备好一块砖：子集保成员
  requires member(x, A)
  requires subset(A, B)
  ensures member(x, B)
{} // 自动归纳可过

lemma subset_trans(A: lset, B: lset, C: lset)
  requires subset(A, B)
  requires subset(B, C)
  ensures subset(A, C)
{
  match A // subset 按第一参数定义 → 证明按 A 分情况
  case Nil => // subset(Nil,C) 按定义 = true, 秒过
  case Cons(head, tail) => {
    assert member(head, B); // ① 拆 requires subset(A,B) 的一层
    assert subset(tail, B); // ② 同上，剩下那半句
    assert subset(B, C); // ③ 拆 requires 的另一条 (其实云里本来就
有)
    assert member(head, C) by { subset_member(head, B, C); } // ④ head ∈ B ∧ B ⊆ C → head ∈ C
    assert subset(tail, C); // ⑤ 归纳假设自动送达：tail 比 A 小
  }
}
```

逐条凭据 (每行“凭什么”)：- 进入 case Cons(head,tail) 时，云里自动有  $A == \text{Cons}(\text{head}, \text{tail})$ 。于是  $\text{subset}(A,B)$  被 Dafny 按定义展开成  $\text{member}(\text{head},B) \ \&\& \ \text{subset}(\text{tail},B)$ ，①②就是把它两半分别点名。- ④：目标  $\text{member}(\text{head}, C)$ ，云里有  $\text{member}(\text{head},B)$  (①) 和  $\text{subset}(B,C)$ ， $\text{subset\_member}$  的三个槽位全对上，调用它就把结论塞进云。by 抽屉只是显式说明这行 assert 靠谁。- ⑤：目标  $\text{subset}(\text{tail}, C)$ 。tail 是 A 的真尾巴——这里 Dafny 的自动归纳出场：它把  $\text{subset\_trans}(\text{tail}, B, C)$  (自身在更小参数上的实例，即归纳假设) 备好了，两个 requires 也都满足 ( $\text{subset}(\text{tail},B)=\text{②}$ ， $\text{subset}(B,C)=\text{③}$ )，所以结论进云。第 12

讲会正式解释”自动归纳”到底怎么回事，现在先当它是既定机制。- Cons 分支结束后，两个 case 都覆盖了目标 → 引理证毕。

**为什么 Dafny 一开始不自动证**（PPT 第 6 页的 subset\_refl 之痛）：目标里出现的新概念（成员）不在云里——云里只有 subset 的定义展开后的一堆 &&，人得”发明”中间引理 subset\_member 搭桥。

## 2.5 subset\_refl：一个”掉进耗子洞”的真实过程（PPT 14–18 页）

要证  $A \subseteq A$ 。手推（PPT 第 15 页，逐字翻译）：

- $A = \text{Nil}$ ：subset(Nil, Nil)，定义第一行给 true ✓。
- $A = \text{Cons}(x, xs)$ ：展开定义—— $\text{subset}(\text{Cons}(x, xs), \text{Cons}(x, xs)) \Leftrightarrow \text{member}(x, \text{Cons}(x, xs)) \ \&\& \ \text{subset}(xs, \text{Cons}(x, xs))$ 。左半边：x 显然在  $\text{Cons}(x, xs)$  里（member 定义第一句  $i == j$ ）✓；右边：**麻烦来了**——需要的不是 subset(xs, xs) 而是 subset(xs, Cons(x, xs))，两个参数不一样！自动归纳给的假设长什么样？形如 subset(xs, xs)（对更小参数实例化同一个命题）。可我们需要的是” $xs \subseteq$  更胖的  $\text{Cons}(x, xs)$ ”。形状对不上，归纳救不了场（PPT 第 17 页：”subset(xs, Cons(x, xs)) does not look like subset(?,?)”——没有任何一对 ?? 能让 IH 正好长成这个）。

于是 PPT 第 18 页尝试”发明小引理”：

```
lemma subset_bigger(x: int, A: lset)
  ensures subset(A, Cons(x, A))
{}    // 期待：自动通过.....实际：☹
```

配了张”从悬崖看风景”的图（Govetts Leap）——玩笑：这个引理数学上为真，但 Dafny 自动证不出来。原因和上面一模一样，而且会套娃：证 subset(Cons(y, ys), Cons(x, Cons(y, ys))) 又需要 subset(ys, Cons(x, Cons(y, ys)))，IH 只给 subset(ys, Cons(y, ys))——还是差那层 Cons(x, ·)。每修一步都冒出新的”参数不一致”，这就是”耗子洞”。

老师文件里最终活下来的方案（subset-casesplits.dfy 第 39–70 行）：

```
lemma append_assoc(x: lset, y: lset, z: lset)
  ensures append(append(x, y), z) == append(x, append(y, z))
{}    // 自动归纳：串接的结合律

lemma subset_append(A: lset, B: lset)
  ensures subset(A, append(B, A))          //  $A \subseteq B++A$ ：对”任何 B”都成立！
{
  match A case Nil => {}
  case Cons(x, xs) =>
    assert member(x, append(B, A));          // ①
    assert subset(xs, append(append(B, Cons(x, Nil)), xs)); // ② IH（参数选胖版）
    assert append(append(B, Cons(x, Nil)), xs) == append(B, Cons(x, xs))
      by { append_assoc(B, Cons(x, Nil), xs); } // ③ 结合律改写
    assert subset(xs, append(B, Cons(x, xs))); // ④ ②+③ 合成
}

lemma subset_refl(A: lset)
  ensures subset(A, A)
{
  match A
  case Nil =>
  case Cons(h, t) => {
```

```

    subset_append(t, Cons(h, Nil)); // 给出 subset(t, append(Cons(h,Nil), t))
    assert append(Cons(h, Nil), t) == Cons(h, t); // 定义直接相等: [h]++t == h::t
  }
}

```

关键洞察 (PPT 没明说、但这是本讲真正的”证明学”): - subset\_bigger 死在”目标里那个多出来的 x 是死的”。而 subset\_append 的陈述里 B 是**任意列表**——一个参数被”放开”成变量, 归纳假设就有得选实例。这就是第 13 讲”加强归纳假设”的预演。- 每一行的凭据: - ①:  $x \in B ++ [x..xs]$ ? 注意此处  $A == \text{Cons}(x, xs)$ ,  $\text{append}(B, A) = B$  后面接上 x 再接 xs。x 当然在里面 (append 把 A 整段原样接上), member 事实由定义 + 归纳送达。- ②: 这是 subset\_append 对更小参数 xs 的自动归纳实例:  $\text{subset}(xs, \text{append}(\text{任意 } B', xs))$ , 取  $B' = \text{append}(B, [x])$ 。合法, 因为  $B'$  是任意列表。- ③: 把  $B' = B ++ [x]$  改写回  $\text{append}(B, [x] ++ xs)$  方向——纯串接结合律, 调 append\_assoc。- ④: ②③等量替换  $\rightarrow \text{subset}(xs, \text{append}(B, \text{Cons}(x, xs))) = \text{subset}(xs, \text{append}(B, A))$ , 正好是 Cons 分支的目标  $\checkmark$ 。- subset\_refl 的 Cons 分支: 调 subset\_append(t, [h]) 得 subset(t, [h]++t); 最后一行 assert 说明  $[h]++t == h::t$  就是 append 定义展开一步, Dafny 自己能看穿。于是目标是 subset(t, Cons(h,t))  $\checkmark$ 。

## 2.6 双重案例拆分 + 【练习】all\_distinct\_nub.dfy (老师跳过没讲)

文件里只有定义, 证明是留白:

```

predicate all_distinct(A: lset) {
  match A
  case Nil          => true
  case Cons(h, t) => !member(h, t) && all_distinct(t) // 头不在尾巴里, 且尾巴也两两不同
}

function nub(A: lset): lset {
  match A
  case Nil          => Nil
  case Cons(x, xs) => if member(x, xs) then nub(xs) // x 在后面重复过  $\rightarrow$  丢掉现在的 x
                    else Cons(x, nub(xs)) // 否则保留
}
// 目标:
lemma all_distinct_nub(A: lset)
  ensures all_distinct(nub(A))

```

nub 的直觉: 从前往后扫, **重复出现只留最后一次** (比如  $[1, 2, 1, 3] \rightarrow \text{nub} = [2, 1, 3]$ , 因为第一个 1 在身后重复了被丢)。

证明思路 (模仿 nub 的结构, 对 A 分情况 + 自动归纳): -  $A = \text{Nil}$ :  $\text{nub} = \text{Nil}$ ,  $\text{all\_distinct}(\text{Nil}) = \text{true} \checkmark$ 。-  $A = \text{Cons}(x, xs)$ : nub 的定义又在 member(x, xs) 上 if 了一次  $\rightarrow$  要**双重分情况** (这就是”Double Case-Split”标题的意思)。

```

// 垫脚石: nub 不会无中生有——nub 里的元素都来自原列表
lemma nub_member(x: int, A: lset)
  ensures member(x, nub(A)) ==> member(x, A)
{} // Dafny 自动归纳: Cons 时展开 nub 定义即可

lemma all_distinct_nub(A: lset)
  ensures all_distinct(nub(A))
{
  match A

```

```

case Nil => {}
case Cons(x, xs) => {
  all_distinct_nub(xs);           // IH: all_distinct(nub(xs))
  if member(x, xs) {
    // nub(A) == nub(xs), 目标就是 IH 本身
  } else {
    // nub(A) == Cons(x, nub(xs))
    // 需要: !member(x, nub(xs)) 且 all_distinct(nub(xs))
    assert !member(x, nub(xs)) by { nub_member(x, xs); }
  }
}
} // 【需你机器验证】逻辑如上; 若最后一行报红, 把报错发我

```

凭据链 (else 分支): 目标展开为  $\text{!member}(x, \text{nub}(xs)) \ \&\& \ \text{all\_distinct}(\text{nub}(xs))$ 。- 右半边 = IH。- 左边用反证式思路:  $\text{nub\_member}(x, xs)$  说”若  $x$  在  $\text{nub}(xs)$  里则  $x$  在  $xs$  里”; 而当前分支的假设正是  $\text{!member}(x, xs)$ , 所以  $x$  不可能在  $\text{nub}(xs)$  里——Dafny 自己会把”假设 + 蕴含 + 否后”这三步串起来。- if  $\text{member}(x, xs)$  的两个分支各自闭合, Cons 情况完。

💡 注意 if 的两个分支里, Dafny 分别白拿  $\text{member}(x, xs)$  /  $\text{!member}(x, xs)$ ——2.2 节说的”带条件进云”。

## 2.7 第二部分自测

1. if  $P \{ \dots \}$  在证明里给云加了什么形状的公式? (答:  $P \implies$  分支内各结论, 不是结论本身。)
2.  $\text{subset\_refl}$  为什么卡住? (答: 展开定义后需要  $\text{subset}(xs, \text{Cons}(x, xs))$ , 与 IH 形状  $\text{subset}(?, ?)$  对不上。)
3.  $\text{subset\_append}$  靠什么脱困? (答: 把”多出来的那截”升格为任意参数 B, IH 才能选实例——第 13 讲主题。)

## 第三部分：全称量词 forall (Lecture 10)

对应代码: `code/lecture10-dafny-code/lecture10/ — simplest-quantifiers.dfy、smallestExists.dfy、divides-forall.dfy、subset-forall.dfy`

### 3.1 量词是什么、为什么它俩”包治百病”

- $\forall x. P(x)$  (Dafny 写 `forall x :: P(x)`): “对每一个  $x$ ,  $P(x)$  成立”。
- $\exists x. P(x)$  (Dafny 写 `exists x :: P(x)`): “至少存在一个  $x$  让  $P(x)$  成立” (不需要知道是哪个, 只要存在)。

PPT 的亚里士多德三段论例子: - “所有人都会死” =  $\forall x. \text{human}(x) \implies \text{mortal}(x)$  - “苏格拉底是人” =  $\text{human}(\text{Socrates})$  - 结论”苏格拉底会死” =  $\text{mortal}(\text{Socrates})$ ——用法就是把  $\forall$  句实例化到 Socrates 身上 (记住”实例化”这个词, 3.6 见)。

### 3.2 一个容易漏掉的点：引理的参数就是藏起来的 $\forall$

PPT 第 28 页: 写

```

lemma member_union(x: int, A: lset, B: lset)
  ensures member(x, union(A,B)) <==> member(x,A) || member(x,B)
{}

```

其实证明的是  $(\forall x)(\forall A)(\forall B)(x \in A \cup B \iff x \in A \vee x \in B)$ 。参数表 = 隐式全称。所以”调用引理传实参”就是”实例化  $\forall$ ”(1.4 节早已在用，只是没点名)。推论 (第 29 页):  
`member_union(3, S, T)`; 合法——对一切成立的东西，对 3 当然成立。

### 3.3 反例、否定、对偶

- 证伪一句  $\forall n. n \bmod 2 = 0$  (“每个数都是偶数”)：只需一个反例 (3)。
- “ $\forall$  是假的”  $\iff$  “ $\exists$  一个反例让 P 为假”：  $\neg(\forall x)P \iff (\exists x)\neg P$ ；反过来  $\exists$  也可定义为  $\neg\forall\neg$ ：  
 $(\exists x)P \iff \neg(\forall x)\neg P$ 。
- 翻译练习 (PPT 18–19 页) ——两条黄金搭配，考试爱考：
  - “所有讲师都友好” =  $\forall x. \text{lecturer}(x) \implies \text{friendly}(x)$ 。 $\forall$  配  $\Rightarrow$  (不是  $x$  是讲师时无所谓，所以用蕴含)。
  - 它的否定 = “存在一个不友好的讲师” =  $\exists x. \text{lecturer}(x) \ \&\& \ !\text{friendly}(x)$ 。 $\exists$  配  $\wedge$ 。
  - $\triangle$  PPT 原话：“If you see an existential with an implication, be very, very, very suspicious!”：  $\exists x. P(x) \implies Q(x)$  几乎总是写错了——因为随便取一个 P 为假的  $x$ ，蕴含式自动成立，这句等于废话。(  $\forall$  配  $\wedge$  只是”少见但合法”，如  $\forall n. 0 \leq n \wedge n < n+1$ 。)
- 限定范围：  $\forall n \in \mathbb{N}. 0 \leq n$  是  $\forall n. n \in \mathbb{N} \Rightarrow 0 \leq n$  的缩写；  $\exists x \in \mathbb{R}. x^2=2$  是  $\exists x. x \in \mathbb{R} \wedge x^2=2$  的缩写。Dafny 里用  $|$  写 (3.5)。
- $\exists!$  “恰好存在一个”：  $\exists! x. P(x) \iff \exists x. P(x) \wedge (\forall y. P(y) \Rightarrow y=x)$  (PPT 第 21 页，了解即可，作业里 Dafny 不直接提供 `exists!` 语法)。

### 3.4 Dafny 的量词语法 (这就是你问的”关键字”本体之一)

表达式形式 (量词写在公式里)：

`forall (变量表) | (前提) :: (体)`  
`exists (变量表) | (前提) :: (体)`

- (变量表)：如  $n: \text{nat}, m: \text{int}$ ，类型可省 (能推断时)。
- | 前提可省。省略时 `forall x :: Q` 读作”所有  $x$  满足  $Q$ ”。
- 带  $|$  时读法因量词而异 (3.3 的黄金搭配)：
  - `forall x | P :: Q` = “所有满足  $P$  的  $x$  都满足  $Q$ ” (内部是  $\Rightarrow$ )；
  - `exists x | P :: Q` = “存在一个  $x$ ，它满足  $P$  并且满足  $Q$ ” (内部是  $\wedge$ )。
- 例子 (PPT 第 23 页，注意它是个假命题演示)： `forall n: nat, m | 2 < n :: 2 * n < n * m`。

#### 【练习】simplest-quantifiers.dfy——量词可以随便待在哪儿

```
predicate foo(n: nat)           // 抽象谓词：不写体，"设 foo 是某性质"

lemma dumb()
  requires forall n: nat :: 0 < n ==> foo(n)    // 假设①：正数都满足 foo
  requires foo(0)                               // 假设②：0 满足 foo
  ensures forall n: nat :: foo(n)              // 目标：所有自然数满足 foo
{
  assert forall n: nat :: foo(n);
}
```

凭据：任取自然数  $n$ 。要么  $n == 0$ ，用假设②；要么  $0 < n$  (自然数非 0 即  $>0$ )，用假设①推出 `foo(n)`。Dafny 自己会做这个 case 判断。💡 量化和普通公式一样能出现在 `requires/ensures/assert` 里，没有什么特殊地位——“quantified formulas are just formulas”。

### 3.5 难点：怎么证一句”藏着”的 $\forall$ ”

PPT 第 34–35 页：assert forall x :: P(x); 有时 Dafny 直接红，因为你没法在证明里提到 **x**——它被  $\forall$  锁在公式内部。Dafny 给了一个专门的**语句**形式（注意：和 3.4 的表达式形式同词不同物）：

```
forall x | 前提 ensures 结论 {  
  ...    // 在这里，x 是“任取的一个”，可以随便提  
}  
        // 跑完这段，forall x | 前提 :: 结论 就进了云
```

- | 前提 可省，也可挪成 forall x ensures 前提 ==> 结论 {...}。
- ⚠ PPT 括号里专门吐槽：不能把 | 写成 requires。

老师文件的演示（univ-recap.dfy 的 somename\_arbitrary）：

```
lemma somename_arbitrary()  
  ensures forall x | 0 < x :: q(x)  
{  
  forall x | 0 < x ensures q(x) { }    // x 任取且知 0<x; 体空 = Dafny 自己搞定 q(x)  
}
```

（q 的定义恰好是 true，所以体能空。）

两个方向的口诀（PPT 第 7–8 页那张表，第四部分还要用，先背）

|           | 作为目标（要证它）                      | 作为假设（云里有）                       |
|-----------|--------------------------------|---------------------------------|
| $\exists$ | 简单：给一个 <b>见证</b> （witness）     | 弱：只能”任取一个”（choose an arbitrary） |
| $\forall$ | 难：“对任意的证”（prove for arbitrary） | 有用：随便 <b>实例化</b> 到谁头上           |

### 3.6 【练习】smallestExists.dfy——Dafny 被量词”逼疯”名场面

```
lemma smallestExists()  
  ensures exists s: nat :: forall m: nat :: s <= m    // "存在一个不超过任何自然数的自然数"  
{  
  assert exists s: nat :: s == 0 && forall m: nat :: s <= m;  
}    // 注释"look at the brown!": 某些版本里会有棕色波浪线 (warning) 而不是红
```

- 数学事实本身为真（s=0）。难点在 Dafny： $\exists$  套  $\forall$  嵌套时，自动推理不知道该替它想谁。
- 修复：直接 assert 带见证的版本（0），并让” $0 \leq m$ ”留给 Dafny 收尾。
- “棕色（brown squiggle）”是 Dafny 的 warning：“这里有量化，我可能证不了/可能超时”的预警，不是错误。PPT 说”Getting Dafny to handle quantifiers well is quite an art... lots of annoying red and brown squiggles in our future!”——以后见怪不怪，红 = 证不过，棕 = 警告可能不靠谱，策略都是：把见证/实例亲手喂给它。

### 3.7 为什么要用量词：subset 的两种证法对比（本讲主线）

第 9 讲的痛：subset\_refl 要绕 append\_assoc 一大圈。第 10 讲引入等价的量化表述：

数学上  $A \subseteq B \iff (\forall x)(x \in A \Rightarrow x \in B)$ 。PPT 第 30 页先提醒：我们早先证过的 member\_subset

```
lemma member_subset(x: int, A: lset, B: lset)  
  requires member(x, A)  
  requires subset(A, B)
```

```

    ensures member(x, B)
  {}

```

把它按 3.2 的规则读回全称式： $\forall x \ A \ B. (x \in A \wedge A \subseteq B) \Rightarrow x \in B$ ——只有  $\Rightarrow$  这一个方向，比  $\Leftrightarrow$  弱。PPT 第 31–33 页的问题（“compare 一下”）答案就是这个“更弱”。于是老师补证完整等价（subset-forall.dfy 第 34–57 行，即【练习】subset\_forall）：

```

lemma subset_forall(A: lset, B: lset)
  ensures subset(A,B) <==> forall x :: member(x,A) ==> member(x,B)

```

老师文件的证明思路（双向，各一个 if 分支）：- 左 $\Rightarrow$ 右（if subset(A,B) 分支）：假设子集成立，再对 forall x 用 3.5 的句式或直接归纳。文件里写得很省——match A 两个 case 空着就过了：Nil 时右边是  $\forall x. \text{false} \Rightarrow \dots$  (vacuous, 前件恒假)  $\checkmark$ ；Cons 时 Dafny 自动归纳 + 定义展开。- 右 $\Rightarrow$ 左（else 分支，即  $\neg \forall \Rightarrow \neg \text{subset}$ ）：!subset(A,B) 时 match A: Nil 不可能 (subset(Nil,B) 恒真)；Cons(h,t) 再 if 一次 !member(h,B)——是则 assert exists x :: member(x,A) && !member(x,B) (见证 h)，否则 t 已破坏子集 (assert !subset(t,B) 送 IH 再要一个存在见证)。两个  $\exists$  见证各让右边的  $\forall$  被打破。双重 case split + 见证，全是前几节的招式。

有了它，subset\_refl 变成三行（文件第 63–72 行）：

```

lemma subset_refl(A: lset) ensures subset(A, A)
{
  match A
  case Nil => {}
  case Cons(h, t) => {
    assert member(h, A); // 显然
    assert forall x: int | member(x, t) :: member(x, A); // t 的成员当然也是 A 的成员 (IH+定义)
    assert subset(t, A) by { subset_forall(t, A); } // 用等价式，把"子集"换成"V 成员"来证
  }
}

```

💡 这就是“expressiveness can be wonderful”（PPT 第 25 页）：同一个事实，写成带  $\forall$  的形式后，证明只需盯“任意元素”，不用再操纵整个列表的结构。

### 3.8 【练习】divides-forall.dfy：素数检查器 $\times \forall$

```

predicate divides(m: nat, n: nat) { // 这一版 divides 用取模定义
  if m == 0 then n == 0 else n % m == 0
}

```

```

predicate primeCheck(t: nat, n: nat) // "t, t-1, ..., 2 都不整除 n"
  requires 1 < n
{
  t <= 1 || (!divides(t, n) && primeCheck(t - 1, n))
}

```

// primeCheck(4, 35) 为 true (4,3,2 都不整除 35)，尽管 35 不是素数——它只查 2..4

老师要证的性质（第 38 页 + 文件里的完整目标）：

```

lemma primeCheck_forall(t: nat, n: nat)
  requires 1 < n
  ensures primeCheck(t, n) <==> forall x: nat | 1 < x <= t :: !divides(x, n)
{ }

```

- 读法：右边是**范围量词**（3.4：  $| 1 < x \leq t$  即前提），意思”对每个满足  $1 < x \leq t$  的自然数  $x$ ， $x$  不整除  $n$ ”。
- 为什么左右等价：primeCheck( $t, n$ ) 展开就是  $t \leq 1 \vee (t | n \wedge \text{primeCheck}(t-1, n))$ ，一路递归展开到  $t \leq 1$ ——正好把  $2..t$  每个数的”不整除”串成合取；而” $\forall$  限定在  $1 < x \leq t$ ”和”逐个合取  $2, 3, \dots, t$ ”是同一件事。Dafny 靠对  $t$  的自动归纳 + 范围量词的定义展开，能自动闭合（所以体可以空  $\{\}$ ；若你的版本报红，用 3.7 的技巧：先 subset\_forall 式地 assert 出  $\forall$ ，或补 assert primeCheck( $t, n$ ) ==> (forall  $x \mid 1 < x \leq t :: !\text{divides}(x, n)$ ) by  $\{\}$  逐方向拆)。
- 顺带说明”为什么不用 % 定义 divides”：第四部分将看到用  $\exists$  定义的版本，能解锁”传递性”这类证明——这是两种定义高下立判的地方。

### 3.9 subset-forall.dfy 后半：列表反转 $\times \forall$

```
function reverse<T>(l: list<T>): list<T> {
  match l
  case Nil          => Nil
  case Cons(x,xs) => append(reverse(xs), Cons(x, Nil))
}
```

```
lemma mem_reverse(x: int, A: lset)
  ensures member(x, reverse(A)) <==> member(x, A)
```

老师的证明用了**外层再套一层 case split** (if member( $x$ , reverse( $A$ ))  $\{\dots\}$  else  $\{\dots\}$ )，内部再 match  $A$  分情况；关键一步和 1.4 节 union 完全同款：assert member( $x, A$ ) by { member\_append( $x$ , reverse( $t$ ), Cons( $h, \text{Nil}$ ))); }——把 reverse(Cons( $h, t$ )) 展开成 append(reverse( $t$ ), [ $h$ ])，再用”成员  $\in$  append”定理拆 &&。正向、反向各一遍。

最后两个 lemma 是本讲的” $\forall$  威力展示”：

```
lemma reverse_subset(A: lset)
  ensures subset(reverse(A), A)
  ensures subset(A, reverse(A))
{
  assert subset(reverse(A), A) by {
    subset_forall(reverse(A), A);           // ① 换成  $\forall$  目标
    forall x: int | member(x, reverse(A)) ensures member(x, A) { // ②  $\forall$  证明语句!
      mem_reverse(x, A);                     // ③ 对”任取且已知反向成员”的  $x$  调用
    }
  }
  // 另一边对称
}
```

三步套路背下来，这是 Dafny 里证” $\forall$  型子集目标”的模板：① 等价改写  $\rightarrow$  ② forall-ensures 语句引入任意  $x \rightarrow$  ③ 在体内对  $x$  调用已知引理。

同一文件末尾 prove1/prove2 是老师的随堂小题（对非空列表 member( $l.h, l$ )、subset( $l.t, l$ )，用的也是 ②③ 模板，可当 3.9 的练习题自己做。

### 3.10 第三部分自测

1.  $\exists x. P(x) \implies Q$  为什么几乎总写错？(PPT：前件取假即真，无信息。)
2. forall  $x \mid P :: Q$  和 exists  $x \mid P :: Q$  展开后分别是？( $\Rightarrow$  与  $\wedge$ 。)
3. 证明” $\forall$  型目标”的两件武器？(引理参数隐式  $\forall$  + forall  $x \dots$  ensures ...  $\{\}$  语句。)

## 第四部分：存在量词 exists——“存在”与 $\exists$ (Lecture 11)

对应代码：code/lecture11-dafny-code/lecture11/ — exists-witness.dfy、divides-trans.dfy、list-nth.dfy、intlist-bound.dfy、univ-recap.dfy 这正是你问的”存在 + 关键字”最核心的一讲。

### 4.1 总纲： $\exists$ 的两种活法（本讲一切由此展开）

3.5 那张表， $\exists$  的两格展开讲：

| 手里有什么                      | 术语                                | 怎么做                                                       |
|----------------------------|-----------------------------------|-----------------------------------------------------------|
| 要证 $\exists x. P(x)$       | provide a <b>witness</b> （给见证）    | assert 一个 $P$ (具体值)，把”谁”交出去—— $\exists$ 只要求存在，不要求多聪明，挑对就行 |
| 已知 $\exists x. P(x)$ （在云里） | <b>choose an arbitrary</b> （任取一个） | <code>var v :  P(v)</code> ；领一个”匿名满足者”出来用                 |

“easy/weak”的评价也来自那张表：证  $\exists$  容易（给个人就行），用  $\exists$  弱（只知它满足  $P$ ，别的一概不知）。

### 4.2 【练习】exists-witness.dfy——给见证的最小样例

```
predicate fitsBetween(x: int, lo: int, hi: int)
  requires lo < hi
{
  lo < x < hi          // 连写: lo<x && x<hi
}

lemma exist1(j: int)
  ensures exists i: int :: fitsBetween(i, 2, 14)
{
  assert fitsBetween(13, 2, 14);
}
```

凭据：目标是”存在  $i$  在 2 和 14 之间”。你 assert 了一条 `fitsBetween(13,2,14)`，即  $2 < 13 < 14$ （Dafny 验算术：真）。云里有了”13 这个具体见证满足性质”， $\exists$  目标立刻达成——**见证法就是”点名”**。作业里证  $\exists$  目标，先问自己：“我能不能直接点一个数/一个列表出来？”能，就 assert 那个点名的式子。

### 4.3 【练习】divides-trans.dfy—— $\exists$ 与整除传递性（本讲王牌例题）

第 10 讲用 `%` 定义 divides；这一讲换成  $\exists$  定义，看看区别：

```
ghost predicate divides(m: nat, n: nat) {
  exists q: nat :: m * q == n    // "m 整除 n" = "存在商 q 使 m·q = n"
}
```

- ghost：见 0.3—— $\exists$  没法算（“找商”是搜索问题），所以这个谓词只许在证明里出现，不出现在可执行代码里。
- 数学目标（PPT 第 19 页）： $m \mid n \wedge n \mid p \Rightarrow m \mid p$ 。老师的完整证明：

```
lemma divides_transitive(m: nat, n: nat, p: nat)
  requires divides(m, n)
```

```

requires divides(n, p)
ensures divides(m, p)
{
  var q1: nat :| q1 * m == n;    // ① 从 requires#1 拆出见证 q1
  var q2: nat :| q2 * n == p;    // ② 从 requires#2 拆出见证 q2
  assert q2 * q1 * m == p;      // ③ 代数:把 ② 的 n 用 ① 替换
  assert exists Q: nat :: Q * m == p; // ④ 给目标 ∃ 点名 Q = q2*q1
}

```

每步凭什么（一步都别跳）：- 云里 `requires divides(m,n)`：Dafny 展开 `divides` 定义 =  $\exists q. m \cdot q == n$ 。:| 的语义（PPT 第 16 页）：**var v :| F(v)** = “Dafny，我相信 F 有解（你得能从云里看出  $\exists v.F(v)$ ），给我起个名字叫 v，并把 F(v) 塞进云”。所以 ① 的合法性 = 云里恰有那条  $\exists$ ；不合法（ $\exists$  不可证）时这一行标红——:| 行红 = “你给的存在性没依据”。- ③： $q2 * n == p$ ，n 换成  $q1 * m$  得  $q2 \cdot (q1 \cdot m) == p$ ，乘法结合律 =  $q2 \cdot q1 \cdot m == p$ 。整数算术 Dafny 自动。- ④：目标 `divides(m,p)` 展开 =  $\exists Q. Q \cdot m == p$ ，把 ③ 的事实（Q 取  $q2 \cdot q1$ ）呈上去。证毕。- 💡 对比第 10 讲：用 % 的定义证传递性要碰整除取模的一堆引理；用  $\exists$  定义，**定义本身就是证明**。“ $\exists$  定义”是把“存在某个东西使其成立”直接写进概念里，之后用起来只需拆见证——量词带来的表达力。

#### 4.4 univ-recap.dfy——instantiate 侧写（对照用）

```

lemma somename_instantiate(y: nat)
  requires forall x | 0 < x :: q(x)
  ensures r(y / 103)
{
  if 0 < y {
    assert q(y);           // ∀ 实例化到 y: 合法因为当前分支已知 0<y
    assert r(y / 103);     // q 定义是 true, r 也是 true, Dafny 看穿
  }
}

```

注意” $\forall$  + |前提”实例化时，前提要当场兑现（这里用 if 分情况拿到  $0 < y$ ）。这是”用  $\forall$  “的标准姿势：想代谁，先证得（或分情况拿到）谁满足前提。

#### 4.5 【练习】list\_nth.dfy——member $\iff \exists \text{index}. \text{nth}$

```

function nth<T>(l: list<T>, n: nat): T
  requires n < length(l)           // 索引必须合法
{
  match l
  case Cons(h, t) => if n == 0 then h else nth(t, n - 1)
  // 没有 Nil case: requires 说 n < length(Nil)=0 不可能, 类型系统知道此分支到不了
}

```

```

lemma mem_nth(i: int, l: list<int>)
  ensures member(i, l) <==> exists n: nat | n < length(l) :: nth(l, n) == i

```

读目标：“i 出现在列表里  $\iff$  存在一个合法下标 n，第 n 个元素是 i”。老师/同学版证明（文件里已填的那版，右 $\rightarrow$ 左 Dafny 自动；左 $\rightarrow$ 右如下逐行拆）：

```

lemma mem_nth(i: int, l: list<int>)
  ensures member(i, l) <==> exists n: nat | n < length(l) :: nth(l, n) == i
{
  match l
  case Nil =>                                     // member(i,Nil)=false; 右边 ∃ 也无 n<0 可取,
  两头同假 ✓

```

```

case Cons(h, t) => {
  if member(i, l) {
    if i == h {
      assert nth(l, 0) == i;           // nth 定义: n=0 取头。见证点名 n=0 ✓
    } else {
      assert member(i, t);             // member(i,Cons(h,t))= i==h||member(i,t),
且 i≠h
      var n: nat :| n < length(t) && nth(t, n) == i; // 拆 IH (对 t 的自动归纳) 给的
      assert nth(l, n + 1) == i;       // nth(Cons(h,t), n+1) = nth(t,n) = i
      assert n + 1 < length(l);       // length(Cons(h,t)) = 1+length(t) ✓
      assert exists index: nat | index < length(l) :: nth(l, index) == i; // 见证
n+1 ✓
    }
  }
  // else 方向 (右→左): Dafny 自己会 (∃ 拆出来直接归纳定义), 文件里留空
}
}

```

凭据链 (else 里的五步): 1. assert member(i,t): 从 member(i,l) = (i==h) || member(i,t) 且 i!=h 取 || 的另一支。2. var n :| ...: 自动归纳给出 IH member(i,t) <==> ∃n..., 左已证 → 右的 ∃ 进云 → :| 拆见证 n。3. nth(l,n+1): 按 nth 定义, n+1≠0 走 else 支 = nth(t,n) = i。4. 长度合法: n+1 < 1+length(t) = length(l)。5. 把 n+1 作为见证 assert 出 ∃ 目标 (| 前提用第 4 步)。💡 这里 ∃ 的两种活法同时出现: 先拆 IH 的 ∃ (:|), 再给自己的 ∃ 目标喂见证 (assert)。一讲一文件全用上了。

## 4.6 【练习】intlist-bound.dfy——“每个整数列表都有上界”

```

predicate exceeds(i: int, l: list<int>) { // i 严格大于列表每个元素
  match l case Nil => true // 空列表: 真空成立
  case Cons(h, t) => i > h && exceeds(i, t)
}
function max(a: int, b: int): int { if a > b then a else b }
function max_list(l: list<int>): int
  requires l != Nil
{
  match l
  case Cons(h, Nil) => h
  case Cons(h, t) => max(h, max_list(t))
}

```

老师文件给的两条证明 (注释说 “will probably need an intermediate lemma”, max\_list\_exceeds 就是那个中间引理):

```

lemma max_list_exceeds(l: list<int>)
  requires l != Nil
  ensures exceeds(max_list(l) + 1, l)
{
  match l
  case Cons(h, Nil) => {} // 单元素: max+1 = h+1 > h ✓ 自动
  case Cons(h, t) => {
    assert max_list(l) + 1 > max_list(t);
    assert max_list(t) + 1 > max_list(t); // t 非空时由定义/IH (见证)
    assert exceeds(max_list(t) + 1, t); // IH: 对更小列表 t 的自动归纳
    assert exceeds(max_list(Cons(h,Nil)) + 1, Cons(h,Nil));
    assert exceeds(max_list(Cons(h,t)) + 1, Cons(h,Nil));
  }
}

```

```

    assert exceeds(max_list(Cons(h,t)) + 1, Cons(h,t)); // 目标
  }
}

lemma boundExists(l: list<int>)
  ensures exists m :: exceeds(m, l)
{
  match l
  case Nil => { assert exceeds(1, l); } // 见证 m=1 (真空)
  case Cons(h, t) => {
    assert exceeds(max_list(l) + 1, l) by { max_list_exceeds(l); } // 见证 m = max+1
  }
}

```

读法：- max\_list\_exceeds 的灵魂是”列表最大值加 1”当然超过每个元素；分情况的每一行是把这个直觉拆成 Dafny 认得的小块（老师注释说这些 assert 是”把 IH 摆到恰当位置”。assert 事实 by { 引理; } (1.5) 第二次登场。- boundExists 两个 case 各自”点名”见证，正是 4.1 表格左上格：证  $\exists$  = 给见证。

**课后小题：PPT 布置的”把量词换个顺序，写出（错的）那个命题”**

原命题  $\forall l. \exists i. \text{exceeds}(i, l)$ （每个列表各自有上界——真）。换序：

```

lemma swapped_is_false()
  ensures exists i: int :: forall l: list<int> :: exceeds(i, l) // 一个数同时超过所有
列表——假!
{ } // 会红：它根本不成立，不该被证出来

```

它假的原因：随便谁声称一个  $i$ ，我取列表  $\text{Cons}(i+1, \text{Nil})$ （或  $[i]$ ）就击穿了它（ $\text{exceeds}(i, [i+1])$  要求  $i > i+1$ ）。这就是 3.3 反例思想在  $\exists \forall$  换序上的应用。作业/考试常考” $\forall \exists$  和  $\exists \forall$  谁强”： $\exists i \forall l$  强于  $\forall l \exists i$ （换序一般不等价）。

## 4.7 第四部分自测

1.  $\text{var } v : | F(v)$ ；两句话语义？（前：需云里有  $\exists v.F(v)$ ，否则这行红；后： $F(v)$  进云， $v$  是个只知道满足  $F$  的匿名者。）
2. 证  $\exists$  目标的三板斧？（直接 assert 点名见证；拆已知  $\exists$  拿见证再加工（divides\_trans）；用  $\exists$  定义概念本身（ghost divides）。）

## 第五部分：归纳法（Lecture 12）

对应代码：code/lecture12-dafny-code/lecture12/ — nat-induction.dfy、list-length-inductions.dfy、tree-inductions.dfy

### 5.1 数学：良基关系与归纳（PPT 第 7–10 页，“温和插叙”）

- **良基关系**：关系  $R$  下，每个元素往下只能走有限步（没有无限下降链  $x_0 R x_1 R x_2 R \dots$  永不停）。“自然数上的  $<$ ”是良基的（往下走几步就撞 0）；“ $\leq$ ”不是（ $\dots 2 \leq 1 \leq 0$  反方向无限）。PPT 举奥运奖牌照（金,银,铜）的字典序也是良基的：金不够就一直降，降无可降就停。
- **通用归纳原理**：若  $R$  良基，且对任取的  $x$ ：
  1. 假设所有  $R$ -比  $x$  小的都有性质  $P$ （= 归纳假设 IH）；

2. 推出  $x$  也有  $P$ ; 那么**所有东西都有  $P$** 。

- 在  $\mathbb{N}$  上特化 = 你中学学的数学归纳法: IH” 对所有  $y < n$  成立” (强归纳), 等价于传统两步” 证  $P(0)$ ; 由  $P(n)$  证  $P(n+1)$  “(PPT 第 9 页说 Believe it or not, 等价)。
- 在树/列表上特化: “是它的子树/子列表” 是良基关系。列表情形: 证 Nil 时, 再对  $\text{Cons}(h,t)$  假设” 性质对  $t$  成立”  $\rightarrow$  对所有列表成立。树情形: 对每个子树假设立足, 对 Node 成立。第 13 讲树证明” 更难” 的根源就在这: 一个 Node 有两个子树 = 两条 IH。

## 5.2 Dafny 与归纳: 自动模式 + 开关

第 8、9 讲其实已经被 Dafny 的自动归纳救过多次 (`member_append`、`subset_member`、`sumUpTo` 的 `triangle`)。它的机制 (PPT 第 18 页, 全课最要紧的一句话):

归纳假设 = 对你正在证明的这个引理、在更小参数上的一次调用。 “To use the I.H., call it! `myresult(e)`; where `e` has to be smaller.”

看穿它的演示: 关掉自动归纳——在引理名前加属性 `{:induction false}`:

```
lemma {:induction false} myresult(...) ensures  $\Phi$  { ... }
```

关了以后, 原来空 `{}` 能过的证明就红了, 你必须**手动**: ① `match`/分情况模仿函数结构; ② 对更小的参数递归调用引理自己 (这次调用就是 IH)。

### 例 1: `nat-induction.dfy`——`sumUpTo` 闭式公式 (手动归纳模板)

```
function sumUpTo(n: nat): nat {
  if n == 0 then 0 else n + sumUpTo(n - 1)
}

lemma {:induction false} sumUpTo_closedForm(n: nat)
  ensures sumUpTo(n) == n * (n + 1) / 2
{
  if n == 0 { assert sumUpTo(0) == 0; }      // 基础情形:  $0 == 0*1/2$  ✓
  else {
    sumUpTo_closedForm(n - 1);                // ★ 归纳假设: 对  $n-1$  的那条公式
  }                                           //  $n + (n-1)n/2 == n(n+1)/2$  算术自动 ✓
}
```

两行证明的凭据: `else` 分支里云有 IH `sumUpTo(n-1) == (n-1)*n/2`; 目标是 `sumUpTo(n) == n(n+1)/2`; 按定义 `sumUpTo(n) = n + sumUpTo(n-1)`, 代入 IH 后只差一个恒等式 `n + n(n-1)/2 == n(n+1)/2`——线性整数算术, Dafny 自动完成 ( $\div 2$  处 Dafny 知道 `n(n+1)` 为偶: 这也在它的自动算术范围内)。

💡 模板化: 分情况 (`n==0` / `n>0`) + 自己调自己 (`n-1`)。下面列表、树的证明全是这个模板换壳。

### 例 2: `sumSquares` (平方和闭式, 同一模板)

```
function sumSquares(n: nat): nat { if n == 0 then 0 else n*n + sumSquares(n-1) }

lemma {:induction false} sumSquares_closedForm(n: nat)
  ensures sumSquares(n) == n * (n + 1) * (2 * n + 1) / 6
{
  if n == 0 {}
  else { sumSquares_closedForm(n - 1); }
}
```

目标展开:  $n^2 + (n-1)n(2n-1)/6 == n(n+1)(2n+1)/6$ 。三次多项式恒等——1.3 节 solve\_poly 的教训说三次 Dafny 可能卡。文件里它却过了: 因为两边通分后是同一个多项式恒等式 (不需要”因式分解”, 只需”展开比对”), 展开比对在自动范围内。卡与不卡一线之隔: **给 Dafny 的是恒等式 (行) 还是因式分解 (不行)**。若你的 Dafny 版本这里红了, 补一行: `assert n * (n+1) * (2*n+1) == (n-1)*n*(2*n-1) + 6*n*n;` (=两边×6 后的展开形式)——见证恒等式, Dafny 负责验。【需验证】

### 5.3 列表归纳: list-length-inductions.dfy

要证的两条 (PPT 第 23 页): 1. `length(append(l1, l2)) == length(l1) + length(l2)` 2. `length(reverse(l)) == length(l)`

**length\_append (文件里声明没写体: 老师留的题, 答案是”真不需要体”)**

```
lemma {:induction false} length_append<T>(l1: list<T>, l2: list<T>)
  ensures length(append(l1, l2)) == length(l1) + length(l2)
```

- **猜对归纳对象** (PPT 第 13 页方法论): 看 `append` 的定义——match 的是**第一个参数**。所以归纳对象是 `l1`。
- 手证: `l1 = Nil`: `append(Nil, l2) = l2`,  $0 + \text{length}(l2) \checkmark$  (定义直接相等)。 `l1 = Cons(h, t)`: `append = Cons(h, append(t, l2))`, 长度 =  $1 + \text{length}(\text{append}(t, l2)) = \text{IH} = 1 + \text{length}(t) + \text{length}(l2) \checkmark$ 。
- 写成 Dafny:

```
{ match l1
  case Nil      => {}
  case Cons(h, t) => { length_append(t, l2); } // IH
}
```

**length\_reverse (文件里有完整证明——注意它先用了 length\_append)**

```
lemma {:induction false} length_reverse<T>(l: list<T>)
  ensures length(reverse(l)) == length(l)
{
  match l case Nil => { assert length(reverse(l)) == length(l); } // 两边同 0 ✓
  case Cons(h, t) => {
    length_reverse(t); // IH: length(reverse(t)) ==
length(t)
    length_append(reverse(t), Cons(h, Nil)); // 用已证引理拆 append
  }
}
```

`Cons` 分支凭什么: `reverse(Cons(h, t)) = append(reverse(t), [h])` (定义), 由 `length_append`: 长度 = `length(reverse(t)) + 1`; IH 把 `length(reverse(t))` 换成 `length(t)`; 目标右边 `length(Cons(h, t)) = length(t) + 1 \checkmark`。

💡 两个引理互相成就: 证 `reverse` 之前必须先有 `append`——先证小的、简单的, 再用它证大的。这个”引理堆叠”在第 13 讲达到巅峰。

### 5.4 树归纳: tree-inductions.dfy

树类型 (第 4 讲就有):

```
datatype tree<V> = Lf | Node(k: int, v: V, tree<V>, tree<V>)
function size<V>(t: tree<V>): nat {
  match t case Lf => 0
  case Node(_, _, lt, rt) => size(lt) + size(rt) + 1
```

```

}
function mirror<V>(t: tree<V>): tree<V> {
  match t case Lf => Lf
  case Node(k,v,lt,rt) => Node(k,v,mirror(rt),mirror(lt)) // 左右交换
}
function keys<V>(t: tree<V>): list<int> {
  match t case Lf => Nil
  case Node(k,_,lt,rt) => append(keys(lt), Cons(k, keys(rt))) // 中序遍历
}

```

### size\_mirror: 镜像不改变节点数

```

lemma {:induction false} size_mirror<V>(t: tree<V>)
  ensures size(mirror(t)) == size(t)
{
  match t case Lf => { assert size(t) == 0; assert size(mirror(t)) == 0; }
  case Node(k, v, l, r) => {
    size_mirror(l); // IH1: size(mirror(l)) == size(l)
    size_mirror(r); // IH2: size(mirror(r)) == size(r)
  }
}

```

Cons 分支凭据:  $\text{mirror}(\text{Node}(k,v,l,r)) = \text{Node}(k,v, \text{mirror}(r), \text{mirror}(l))$ ,  $\text{size} = \text{size}(\text{mirror}(r)) + \text{size}(\text{mirror}(l)) + 1$ ,  $\text{IH}_1 + \text{IH}_2$  换成  $\text{size}(r) + \text{size}(l) + 1 = \text{size}(t)$

✓。 “Why trees are worse” 第一次现身：一次要用两条 IH（文件注释还提醒：递归调用参数必须严格更小，否则归纳无意义）。

### length\_keys: keys 的长度 = 节点数

目标  $\text{length}(\text{keys}(t)) == \text{size}(t)$ 。文件里这条证明最长（第 57–76 行），Node 分支的招是把三块拼成两块：

```

case Node(k, v, l, r) => {
  length_keys(l); // IH1: length(keys(l)) == size(l)
  length_keys(r); // IH2: length(keys(r)) == size(r)
  assert length(keys(l)) == size(l);
  assert length(keys(Node(k, v, Lf, r))) == size(Node(k, v, Lf, r)); // IH1/IH2 用在“半棵树”上
  assert length(keys(Node(k,v,Lf,r))) + length(keys(l)) == size(l) +
size(Node(k,v,Lf,r));
  assert size(t) == size(l) + size(Node(k, v, Lf, r));
  assert keys(t) == append(keys(l), keys(Node(k, v, Lf, r)));
  assert length(keys(t)) == length(keys(l)) + length(keys(Node(k,v,Lf,r))) by {
    length_append(keys(l), keys(Node(k, v, Lf, r))); // ★ 借 5.3 的引理拆 append
  }
}

```

关键观察：Node(k,v,Lf,r) 是一棵真子树吗？是——它是 t 的 Node 的右子再包一层？不！注意它和 t 不是子树关系。文件敢对 IH<sub>2</sub> 用在它身上，靠的是 Dafny 的自动归纳给的是“所有真子树”的 IH（5.1: size(mirror/keys) 这类函数匹配在子结构上），Node(k,v,Lf,r) 是 t 的真子树（ $t = \text{Node}(k,v,l,\cdot)$ ，r 是子树，Lf 是子树，但 Node(k,v,Lf,r) 整体……它其实不是子树，而是“把 l 换成 Lf”得到的更小结构）。这里用的是 5.2 的“自己调自己在更小参数”：length\_keys(Node(k,v,Lf,r)) 是合法递归调用（size 严格小 1），所以第三条 assert 是对这个半截树的 IH<sub>1</sub>——第三行 assert 把两条 IH 相加，第四行把 size(t) 拆成同一拆分，第五、六行用 keys/append 的定义缝合。整个分支 = “用两棵子树的 IH + 一棵‘替换掉左子’的树的 IH + length\_append”。🌶️ 这正是 PPT 第 44 页（第 13 讲）预告的：“树的 IH 会被用在各

种没见过的中间形状上”——手动归纳时，这种“半棵树”技巧是树证明的分水岭，看懂这条链 = 树归纳毕业。

(自动模式下——去掉 `{:induction false}`——这些 Dafny 自己就做，PPT 第 16 页 “Good to know that this is what Dafny does for us”。)

## 5.5 第五部分自测

1. 怎么“使用”归纳假设？(对更小参数调用引理自身。)
2. 怎么强制手写归纳？(`{:induction false}`。)
3. 列表/树归纳的证明骨架长什么样？(match 模仿函数 + 每个递归 case 里调用自己一次/两次。)

---

## 第六部分：吓人的归纳——加强归纳假设 (Lecture 13)

对应代码：code/lecture13-dafny-code/lecture13/ — hanoi.dfy、maxlist.dfy、tree-flatten.dfy 你缺席的一讲。它的全部技巧只有一句话：“pin 死变化的参数会让 IH 没得用；把它放开，证一个对所有取值都成立的更强命题。”

### 6.1 本讲地图

- accumulator (累加器) 风格：给函数多加一个参数，沿途攒结果 (把  $O(n^2)$  变  $O(n)$ )。函数式里 `f(..., acc)`，命令式里等价于 while 循环里改 acc 变量。
- append 的 l2 参数全程不变 → 归纳轻松；reverse(l, acc) 的 acc 每步在变 → 归纳变难。难不在函数，在证明。
- PPT 警告句，出现在作业评语里你要认识：“You need to strengthen the inductive hypothesis” (你需要加强归纳假设)。

### 6.2 例 1：汉诺塔 (hanoi.dfy, 完整讲解)

规则：三根柱子、大盘不能压小盘、一次移一盘。问  $n$  个盘从 1 号柱移到 3 号柱要几步。

#### 两种写法

```
function hanoi1(n: nat): nat {           // "自然"写法：直接报步数
  if n == 0 then 0 else 1 + 2 * hanoi1(n - 1)
}
function hanoi2(n: nat, a: nat): nat {   // 累加器写法：a = 之前已花掉的步数
  if n == 0 then a else hanoi2(n - 1, 2 * a + 1)
}
```

hanoi1 的递推就是 PPT 第 13 页策略：搬上面  $n-1$  个 ( $h(n-1)$  步) → 搬最大盘 (1 步) → 再搬  $n-1$  个 ( $h(n-1)$  步) →  $h(n) = 2h(n-1)+1$ ,  $h(0)=0$ 。hanoi2 的思路 (PPT 第 16-17 页, “反过来数”)：手上已经攒了  $a$  步，接下来每把“上面一摞”重新搬到新盘上，成本翻倍加一 ( $2a+1$ )； $n$  个盘搬完时  $a$  就是答案。

#### 卡壳点 (PPT 第 19-22 页) ——手动推给你看

想证 `hanoi1(n) == hanoi2(n, 0)` (文件里 hanoi12, PPT 说 Dafny 证不出, 事实: 空 {} 时 Cons 情况红)。对  $n$  归纳:  $n=0$  两边同 0 ✓。设  $n>0$ ,  $IH = hanoi1(n-1) == hanoi2(n-1, 0)$ 。展开两边:

左边  $\text{hanoi1}(n) = 1 + 2 \cdot \text{hanoi1}(n-1) = 1 + 2 \cdot \text{hanoi2}(n-1, 0)$  (用 IH, 把  $\text{hanoi2}$  的第二参数钉在 0)

右边  $\text{hanoi2}(n, 0) = \text{hanoi2}(n-1, 2 \cdot 0 + 1) = \text{hanoi2}(n-1, 1)$

要合上得证  $1 + 2 \cdot \text{hanoi2}(n-1, 0) == \text{hanoi2}(n-1, 1)$ ——IH 只给了  $a=0$  的信息, 可  $a=1$  的实例我们没有。这就是”第二参数不匹配, 卡住”。病根 (PPT 第 22、29 页): 递归调用里两个参数都在变 ( $n \rightarrow n-1, a \rightarrow 2a+1$ ), 把  $a$  钉死在初值 0 的命题弱到不配当自己的归纳假设——“to run  $\text{hanoi2}$  you need steps where  $a \neq 0$ ”。

### 开方: 加强命题

把  $a$  放开, 对一切  $a$  证:

```
lemma hanoi_gen(n: nat, a: nat)
  ensures hanoi2(n, a) == hanoi1(n) + a * (hanoi1(n) + 1)
{} // Dafny 自动通过! (对  $n$  归纳时,  $a$  仍是任意参数  $\rightarrow$  IH 对所有  $a$  可用)
```

- 这个更强的陈述怎么想出来的? PPT 第 23–24 页: 瞪数字。  $n=3$  一列: 7, 15, 23, 31 ( $a=0..3$ ) ——等差 8, 而  $8 = \text{hanoi1}(3)+1$ ;  $n=4$ : 15, 31, 47, 63, 等差 16 =  $\text{hanoi1}(4)+1$ 。规律  $\text{hanoi2}(n, a) = \text{hanoi1}(n) + a \cdot (\text{hanoi1}(n)+1)$ 。💡 “guess the relation from a few values” (文件注释同款): 这是证明工地上真正的”人肉创造力”, Dafny 代劳不了——考试也不会要求你凭空发明, 通常给你数字表让你找规律。
- 为什么这次自动过了? 3.2 节铁律: 引理参数隐式  $\forall$ 。IH 现在是  $\forall a. \text{hanoi2}(n-1, a) = \dots$  ——想要  $a=1$ ? 随便实例化! 卡点消失。PPT 第 27 页点题: “Dafny inducts over a lemma’s parameters, and those are implicitly universally quantified... in particular for  $2^*a+1$ ”。
- 原命题是  $a=0$  的推论:

```
lemma hanoi2_again(n: nat)
  ensures hanoi1(n) == hanoi2(n, 0)
{ hanoi_gen(n, 0); } // 代入  $a=0: \text{hanoi2}(n, 0) = \text{hanoi1}(n) + 0 * (\dots) = \text{hanoi1}(n) \checkmark$ 
```

- 喜欢看见  $\forall$  的写法 (PPT 第 28 页等价版):

```
lemma hanoi_gen2(n: nat)
  ensures forall a: nat :: hanoi2(n, a) == hanoi1(n) + a * (hanoi1(n) + 1)
{}
```

💡  $\text{hanoi\_gen}$  和  $\text{hanoi\_gen2}$  是同一句话的两种写法 (隐式  $\forall$  vs 显式 forall)。文件里两种都给了, 能自动过。

### 警报器 (PPT 第 29–30 页, 本讲最该背的元认知)

递归调用里”累加器变了” = 警报响。凡是函数形如  $f(n, \text{acc})$  且递归调  $f(n', F(\text{acc}))$  ( $\text{acc}$  被改写), 任何”把  $\text{acc}$  钉在初值”的命题都不够归纳用; 要么把命题加强成”对任意  $\text{acc}$ ”, 要么把  $\text{acc}$  的含义讲清楚。

### 6.3 例 2: 【练习】maxlist.dfy——老师课上没做完的题 (文件里挂着 == ???)

```
function max(i: int, j: int): int { if i < j then j else i }
```

```
function maxList(l: list<int>): int // v1"自然"写法: 需要 l 非空
  requires l.Cons?
{
  match l
  case Cons(j, Nil) => j
```

```

    case Cons(j, js) => max(j, maxList(js))
}
function maxA(l: list<int>, maxSoFar: int): int {    // v2 累加器写法: 无前提
    match l case Nil => maxSoFar
    case Cons(j, js) => maxA(js, max(j, maxSoFar))
}
function maxList2(l: list<int>): int                // v1 的皮, v2 的芯
    requires l.Cons?
{
    match l case Cons(j, js) => maxA(js, j)        // 头先入袋, 尾巴用 maxA 扫
}

```

文件里留白的目标:

```

lemma maxA_thm(l: list<int>, A: int)
    ensures maxA(l, A) == ???
lemma finishing_up(l: list<int>)
    requires l.Cons?
    ensures maxList(l) == maxList2(l)

```

按 6.1 的读法先”讲清楚 acc 是什么”:  $\text{maxA}(l, A)$  = “把 A 当作’目前已知的最大值’, 扫完 l 后的最终最大值” =  $\text{max}(l \text{ 中所有元素 与 } A)$ 。把它写死成不依赖空列表的公式:

```

lemma maxA_thm(l: list<int>, A: int)
    ensures maxA(l, A) == if l == Nil then A else max(maxList(l), A)
// 对 l 自动归纳:
// Nil: 左=右=A ✓
// Cons(j,js): 左 = maxA(js, max(j,A)); IH 把它变成 (js 空时) max(j,A),
//              (js 非空时) max(maxList(js), max(j,A)) = max(max(j,maxList(js)), A)
(max 结合, Dafny 自动)
//              右 = max(maxList(Cons(j,js)), A) = max(max(j, maxList(js)), A) ✓
{
}

lemma finishing_up(l: list<int>)
    requires l.Cons?
    ensures maxList(l) == maxList2(l)
{
    match l case Cons(j, js) => {
        maxA_thm(js, j);        // maxA(js,j) = (js空 ? j : max(maxList(js), j))
        // maxList(Cons(j,js)) = (js空 ? j : max(j, maxList(js))); 两式逐 case 相等, max 对
称 ✓
    }
} // 【需你机器验证】逻辑链如上; PPT 还提示“引理参数顺序影响自动归纳选谁”—l 放第一位

```

💡 这就是 hanoi 的镜像练习: 强命题  $\text{maxA\_thm}$  自己可归纳, 原命题  $\text{finishing\_up}$  是它的特例。

### 6.4 例 3: 树摊平 keys/keys2 (tree-flatten.dfy, 本讲压轴)

第 5.4 节 keys 的写法每层调 append, “会把左子树整段再抄一遍”(PPT 第 36 页: 树向左歪时复杂度爆  $O(n^2)$ )。累加器版只用 Cons:

```

function keysA<V>(t: tree<V>, acc: list<int>): list<int> {
    match t case Lf => acc
    case Node(k, _, lt, rt) => keysA(lt, Cons(k, keysA(rt, acc)))
}
function keys2<V>(t: tree<V>): list<int> { keysA(t, Nil) }

```

// 目标 keys2\_keys0 : ensures keys2(t) == keys(t) — 直接证会红 (文件注释: "not provable as it stands")

**acc 的含义** (PPT 第 38–39 页, “把累加器翻译成一句人话”): 读  $\text{keysA}(t, \text{acc}) =$  “**t 的所有键, 按序, 接在 acc 前面**”。两个 case 恰好都在干这事: - Lf: 没有键, 答案就是 acc; - Node: 顺序应为 lt 的键  $\rightarrow k \rightarrow$  rt 的键  $\rightarrow \text{acc}$ 。先造好后半段  $\text{Cons}(k, \text{keysA}(\text{rt}, \text{acc}))$  (= k 接在” rt 的键+acc” 前), 再交给  $\text{keysA}(\text{lt}, \dots)$  让 lt 的键接在更前——严丝合缝。

**这句话就是强引理** (PPT 第 40 页 “The Sentence Is the Lemma”):

```
lemma append_assoc<T>(a: list<T>, b: list<T>, c: list<T>)
  ensures append(a, append(b, c)) == append(append(a, b), c)
{}

lemma append_nil<T>(l: list<T>) ensures append(l, Nil) == l {} // 两条 append 事实
Dafny 自动

lemma keysA_keys<V>(t: tree<V>, acc: list<int>)
  ensures keysA(t, acc) == append(keys(t), acc)
{
  match t
  case Lf => {} // keys(Nil...) 两边都 = acc ✓
  case Node(k, _, lt, rt) => {
    // 目标: keysA(lt, Cons(k, keysA(rt, acc))) == append(keys(lt) ++ [k] ++ keys(rt), acc)
    // 左边: IH_lt 用在 acc' = Cons(k, keysA(rt, acc))  $\rightarrow$  keys(lt) ++ Cons(k, keysA(rt, acc))
    // 里面: IH_rt 用在 acc  $\rightarrow$  Cons(k, keys(rt) ++ acc)
    // = keys(lt) ++ (Cons(k, keys(rt)) ++ acc) (Cons 对 ++ 的定义)
    // = (keys(lt) ++ Cons(k, keys(rt))) ++ acc  $\leftarrow$  ★ 就差结合律这一手:
    append_assoc(keys(lt), Cons(k, keys(rt)), acc);
  }
}
```

```
lemma keys2_keys<V>(t: tree<V>)
  ensures keys2(t) == keys(t)
{
  keysA_keys(t, Nil); // keysA(t, Nil) == append(keys(t), Nil)
  append_nil(keys(t)); // append(..., Nil) == 原列表  $\rightarrow$  keys2 = keys ✓
}
```

PPT 第 42 页的步推链条 (老师课上大概率是念这页的) 我誊出来, 每行标注凭据:

|                                                             |                                                                              |
|-------------------------------------------------------------|------------------------------------------------------------------------------|
| $\text{keysA}(t, a)$                                        |                                                                              |
| $= \text{keysA}(l, \text{Cons}(k, \text{keysA}(r, a)))$     | def'n (keysA 对 Node 展开)                                                      |
| $= \text{keys}(l) ++ \text{Cons}(k, \text{keysA}(r, a))$    | I.H. for l, 用在 $\text{acc}' = \text{Cons}(k, \text{keysA}(r, a)) \leftarrow$ |
| 两处“用在哪儿”                                                    |                                                                              |
| $= \text{keys}(l) ++ \text{Cons}(k, \text{keys}(r) ++ a)$   | I.H. for r, 用在 a                                                             |
| $= \text{keys}(l) ++ (\text{Cons}(k, \text{keys}(r)) ++ a)$ | def'n of ++ (Cons 头挂进 ++ 的定义方向)                                              |
| $= (\text{keys}(l) ++ \text{Cons}(k, \text{keys}(r))) ++ a$ | associativity (★ append_assoc : 唯一需要的外来事实)                                   |
| $= \text{keys}(t) ++ a$                                     | def'n (keys 对 Node 展开, 逆方向读)                                                 |

**为什么树更难** (PPT 第 44–45 页): 两条 IH (左右子树各一) 而且用在不同的累加器值上——l 那条用在  $\text{Cons}(k, \text{keysA}(r, \text{acc}))$ , r 那条才用在 acc。钉死 acc 的命题在这种局面下必死; 只有”对所有 acc 成立”的强命题才供得起两次不同实例化。

## 6.5 本讲的”寓意” (PPT 第 46 页)

归纳证明本身不难——步骤严丝合缝、Dafny 通常还能代劳；**难的是找对那句话**（命题陈述）——要在所有参数之间建立关系，靠”瞪数字/讲故事/找模式”的人味创造力。

三条路标回顾：hanoi（瞪数字 → 猜关系）；maxlist（讲清 acc 语义）；tree-flatten（一句人话直接抄成引理）。

---

## 第七部分：关键字总表 & 练习文件地图

### 7.1 这几讲 Dafny 关键字/语法面孔总表（“老师 PPT 里格式多变”的答案）

**证明结构** | 写法 | 作用 || — | — || `lemma name(参数)` | 定义命题+证明；参数隐式  $\forall$ ，无返回类型 || `requires R` | 假设（内部白拿）/ 调用义务（外部要证） || `ensures E` | 目标（内部要证）/ 调用后送入云的事实 || `{ }` | 证明体；自动成功时可空 || `assert  $\phi$ ;` | 自查  $\phi$  可由云推出；推出则  $\phi$  入云 || `assert  $\phi$  by { 证明 }` | 带抽屉的 assert (1.5) || `ghost predicate` | 只用于推理的谓词（ $\exists$  型定义必须 ghost） |

**分支** | 写法 | 语义 || — | — || `if P { ... } else { ... }`（证明内） | 分支内白拿  $P / \neg P$  || `match x case ... => { ... }`（证明内） | 分支内白拿  $x == \text{该形状}$ ；覆盖全形状 = 证毕 |

**量词（面孔最多，重点看）** | 面孔 | 类别 | 读法 || — | — | — || `forall x :: Q` | 表达式 | 一切  $x$  满足  $Q$  || `forall x: nat :: Q` | 表达式 | 一切自然数  $x$  满足  $Q$  || `forall x: nat \ P :: Q` | 表达式 | 满足  $P$  的一切  $x$  满足  $Q$ （内部  $\Rightarrow$ ） || `exists x: nat \ P :: Q` | 表达式 | 存在  $x$  同时满足  $P$  和  $Q$ （内部  $\wedge$ ） || `forall x \ P ensures Q { 证明 }` | **语句** | 任取  $x$ （白拿  $P$ ），去证  $Q$ ——体内  $x$  可提 || `var v: T \ F(v);` | **语句** | 从云里的  $\exists$  拆出一个匿名见证  $v$ ， $F(v)$  入云 | `ensures forall a :: ...` | 引理参数隐式  $\forall$  的显式版 (6.2 hanoi\_gen2) ||

**终止/归纳** | 写法 | 作用 || — | — || `decreases e` | 用度量  $e$  变小来保证终止（chunks） || `{:induction false}` | 关掉自动归纳，强制手写（12 讲） || 引理自身在更小参数上的调用 | 就是”使用 IH”（5.2 铁律） |

**其他** | 写法 | 作用 || — | — || `datatype / type 别名` | 定义形状 / `type lset = list<int>` || `l.Cons? / l != Nil` | 形状判定 || 抽象 predicate `foo(n:nat)` | 无定义的占位谓词（dumb 例） || `lo < x < hi` | 连写不等式 ||  $\tau$ （如 `list<T>`, `tree<V>`） | 泛型参数 |

## 7.2 练习文件 × 讲解对照（老师讲过的/跳过的）

| 文件                                   | 课堂状态                      | 本教程     |
|--------------------------------------|---------------------------|---------|
| lecture08/lemma-parts.dfy            | 讲（沙盒）                     | 1.2     |
| lecture08/assert-proofs.dfy          | 讲                         | 1.3     |
| lecture08/setlists.dfy               | 讲                         | 1.4     |
| lecture08/chunks-terminates.dfy      | 讲                         | 1.6     |
| lecture09/subset-casesplits.dfy      | 讲                         | 2.4、2.5 |
| lecture09/all_distinct_nub.dfy       | 只给定义，证明留白                 | 2.6 补全  |
| lecture10/simplest-quantifiers.dfy   | 讲                         | 3.4     |
| lecture10/smallestExists.dfy         | 讲                         | 3.6     |
| lecture10/divides-forall.dfy         | 讲（体留空）                    | 3.8     |
| lecture10/subset-forall.dfy          | 讲（含 mem_reverse、prove1/2） | 3.7、3.9 |
| lecture11/exists-witness.dfy         | 讲                         | 4.2     |
| lecture11/univ-recap.dfy             | 讲                         | 4.4     |
| lecture11/divides-trans.dfy          | 讲                         | 4.3     |
| lecture11/list-nth.dfy               | 部分讲（ $\exists$ 方向留白）      | 4.5     |
| lecture11/intlist-bound.dfy          | 讲（换序题布置了没讲）               | 4.6 附答案 |
| lecture12/nat-induction.dfy          | 讲                         | 5.2     |
| lecture12/list-length-inductions.dfy | length_append 没写体         | 5.3 补全  |
| lecture12/tree-inductions.dfy        | 讲（长证明）                    | 5.4 逐行  |
| lecture13/hanoi.dfy                  | 缺席                        | 6.2 全   |
| lecture13/maxlist.dfy                | 缺席 + ??? 留白               | 6.3 补全  |
| lecture13/tree-flatten.dfy           | 缺席                        | 6.4 全   |

### 7.3 常见红/棕色波浪线诊断手册

| 症状                                           | 高频原因                       | 处方                                                         |
|----------------------------------------------|----------------------------|------------------------------------------------------------|
| <code>ensures</code> 整条红, <code>{}</code> 为空 | 云和目标差得远                    | 找洞察 <code>assert</code> 一条中间事实; 或调用现成引理                    |
| <code>match</code> 某 case 内红                 | IH 形状不对                    | 分情况模仿函数定义; 看是否该加强参数 (6.1)                                  |
| 函数红: <code>decreases</code> 不满足              | 递归参数 Dafny 看不出变小           | <code>decreases length(l) + assert 变小 by {定理} (1.6)</code> |
| 调用引理那行红                                      | <code>requires</code> 义务没还 | 在调用前 <code>assert</code> 出前提                               |
| <code>var v :  ...</code> 红                  | 云里没有对应的 $\exists$          | 检查被拆的 $\exists$ 是否真在云中 (定义没展开? IH 没调?)                     |
| 棕色 warning ( <code>exists/forall</code> 附近)  | 量化嵌套, 求解器没把握               | 手工喂见证/实例 (3.6)                                             |

### 7.4 接下来怎么练 (一周计划建议)

1. 今天: 把 1.3、1.4 的每个 lemma 亲手敲进 VS Code, 故意删掉某行 `assert` 看它变红, 再加回来——练”云”的手感。
2. 第 2、3 天: 手抄 2.4 `subset_trans`  $\rightarrow$  合上教程默写; 做 2.6 `nub` (我给的证法, 自己敲一遍并观察 `if` 分支白拿的假设)。
3. 第 4 天: 量词周: 3.4 `dumb`、3.6 `smallestExists`、4.2 `exist1` 各默写; 重点把 7.1 量词表 7 种面孔各造一个自己的例子。
4. 第 5 天: 归纳: 把 `{:induction false}` 加到 `member_append` 上, 亲手补 `match` + 自调用; 做 `length_append` 留白。
5. 第 6 天: 汉诺塔: 只带 6.2 那张数字表, 自己推”猜测关系”并写 `hanoi_gen`; 然后做 `maxlist`。
6. 随时: 卡住的报错原文发给我, 我们对着云逐行查。

—— 完 ——